ELSEVIER

Operations Research Letters 16 (1994) 79–86

operations research letters

Scheduling with batching: minimizing the weighted number of tardy jobs

Dorit S. Hochbaum[a, 1, *], Dan Landy[b, 2]

[a]*IEOR Department and School of Business Administration, University of California, Berkeley, CA 94720, USA*
[b]*IEOR Department, University of California, Berkeley, CA 94720, USA*

(Received 2 September 1993; revised 1 May 1994)

Abstract

The weighted tardiness with batching (WTB) problem can be briefly described as follows: given n jobs, each with a processing time, a weight, and a due date, and given a batch setup time, find a sequence of batches that minimizes the weighted number of tardy jobs. We show that the decision version of WTB is NP-complete, but that it can be solved in pseudo-polynomial time by a dynamic programming algorithm. We also analyze various restricted versions of the problem generated by requiring that one or more of the job parameters (weight, processing time, or due date) is the same for all jobs. For each such version of the problem, we prove NP-completeness or provide a polynomial algorithm.

Key words: Semiconductor manufacturing; Pseudo-polynomial algorithms; Batch scheduling

1. Introduction

In this paper we introduce the *weighted tardiness with batching* (WTB) problem, an instance of which is defined by a batch setup time and a set of n jobs, where each job has a processing time, a weight, and a due date. The objective is to schedule the jobs in batches so as to minimize the weighted number of tardy jobs (jobs completed after their due dates). What distinguishes the problem from other scheduling problems with setup times (see for example [3]) is that the completion time of a job is equal to the completion time of its batch, so that all the jobs together in a batch are completed at the same time. One example of the applicability of this model to a particular manufacturing scenario is described in [6]. Jobs coming off a machine are placed on a pallet; they are then transported in a batch to their next destination, where they will arrive simultaneously. The time required to process and transport the jobs is therefore determined by the time required to set up a new pallet plus the sum of the processing times of all the jobs in the batch.

* Corresponding author. email: dorit@hochbaum.berkeley.edu.

[1] Supported in part by the Competitive Semiconductor Manufacturing project by the Sloan Foundation and by the Office of Naval Research under grant N00014-91-J-1241.

[2] supported in part by the Competitive Semiconductor Manufacturing project by the Sloan Foundation.

SSDI 0167-6377(94)00028-5

Our motivation for studying this problem arose with a problem of determining batch size of wafer lots to run through photolithographic machines. When all dies produced are identical they require the same set of layers, and the same set of masks to be placed on the machines. When a batch is finished, the first mask for the new batch has to be placed. The total time required to set up the masks is the setup time of the batch.

The WTB problem is closely related to the problem of sequencing jobs to minimize tardy task weight, which is NP-complete [7] but which can be solved in pseudo-polynomial time [8]. The WTB problem is also closely related to the following scheduling problem, which has been studied by several researchers: given n jobs, each with processing time p_i and weight w_i, $i \in \{1,\dots,n\}$, and given a batch setup time Δ, find a sequence of batches that minimizes the weighted sum of job completion times. Albers and Brucker [1] showed that the general version of this problem is NP-complete, but there are polynomial algorithms for special cases, such as when all job weights are equal [5], when both weights and processing times are equal [4, 9, 10], and when the job sequence is predetermined [2].

In the following sections we show that the decision version of WTB is NP-complete, but that it can be solved in pseudo-polynomial time by a dynamic programming algorithm. We also analyze various restricted versions of the problem generated by requiring that one or more of the job parameters (weight, processing time, or due date) is the same for all jobs. When all job due dates are equal, the problem remains NP-complete. When either all the job processing times are equal, or all the job weights are equal, the problem is solvable in strongly polynomial time.

1.1. List of problems addressed

WTB: The general problem,
WTB with equal weights,
WTB with equal processing times,
ADEW: Agreeable due dates, equal weights,
ADEP: Agreeable due dates, equal processing times.

2. The WTB problem: The general case

The WTB problem is defined as follows: given n jobs numbered $1,\dots,n$, each with processing time $p_i \in \mathbb{Z}^+$, weight $w_i \in \mathbb{Z}^+$, and due date $d_i \in \mathbb{Z}^+$, $i = 1,\dots,n$, and a batch setup time $\Delta \in \mathbb{Z}^+$, find a schedule that minimizes the weighted number of tardy jobs. A *schedule* B is a sequence of batches $B = (b_1, b_2, \dots, b_k)$, where each batch b_i is a set of jobs. The completion time of batch b_i is given by

$$i\Delta + \sum_{j \in b_k,\ 1 \leqslant k \leqslant i} p_j.$$

The completion time of job j, denoted by t_j, is equal to the completion time of batch b_k when $j \in b_k$. If $t_j \leqslant d_j$, job j is *on-time*; otherwise, it is *tardy*. The cost of schedule B, $F(B)$, is the weighted number of tardy jobs

$$F(B) = \sum_{j:t_j > d_j} w_j.$$

The objective in the WTB problem is to find B to minimize $F(B)$. Given an instance of the WTB problem and an integer $W \geqslant 0$, the decision version of the problem is to answer the question, "Does there exist a schedule in which the weighted number of tardy jobs is at most W?"

Since we are only concerned with jobs that are not tardy, in the remainder of this paper we will use the word *schedule* to indicate the sequence of batches containing the on-time jobs. How the tardy jobs are completed has no effect upon the value of the objective function.

A special case of the WTB problem is when the batch setup time $\Delta = 0$. In this case the problem reduces to one of sequencing n jobs to minimize the tardy task weight, which is an NP-complete problem [7]. In fact, even when all job due dates are equal, minimizing the tardy task weight is NP-complete. This can be proved with a very straightforward reduction from the knapsack problem, where job processing times and weights are set equal to the sizes and values of the knapsack elements, respectively, and each job has a due date equal to the size of the knapsack. Thus we have the following result.

Lemma 1. *The decision version of the WTB problem is* NP *-complete, even when all jobs have the same due date.*

Proof. The proof is by a straightforward reduction from the knapsack problem. □

Although WTB is NP-complete, because the reduction is from a weakly NP-complete problem (knapsack) we might hope to find a pseudo-polynomial algorithm for WTB. Indeed, the following lemma is the basis for such a result.

Lemma 2. *There exists an optimal solution to the WTB problem in which all on-time jobs are in earliest due date (EDD) order. That is, for any two jobs x and y which are in batches i and j, respectively, where $i < j$, we have $d_x \leqslant d_y$.*

Proof. Let $S = (b_1, \ldots, b_k)$ be an optimal schedule for the WTB problem, and suppose S includes jobs x and y in batches b_i and b_j respectively, where $i < j$ and $d_x > d_y$. If job x is moved from batch b_i to batch b_j, all on-time jobs remain on-time, as follows: the only job with an increased completion time is x, which now has completion time $t_x = t_y$, since both jobs are in the same batch. Since y is completed on time, we have

$$t_x = t_y \leqslant d_y < d_x,$$

and thus x is completed on-time. Any schedule can therefore be transformed so that all on-time jobs are in order of increasing due date. □

Lemma 2 is the basis for the following dynamic programming procedure for solving the WTB problem. The jobs are considered in order of increasing due date. Suppose we are given a partial schedule for jobs $1 \ldots j-1$, in which some subset of jobs $\{i_1 \ldots i_k\}$ is on-time. The last batch of this partial schedule will contain job i_k, and possibly some other jobs. Let d be the earliest due date in this last batch, and let t be the completion time of the batch (and hence of the partial schedule). There are now 3 choices for scheduling job j:

1. Add j to the last batch of the current partial schedule, if it will be completed by the earliest due date in that batch (i.e. if $t + p_j \leqslant d$).
2. Add j to a new batch at the end of the current partial schedule, if it will be completed before its due date (i.e. if $t + \Delta + p_j \leqslant d_j$).
3. Let j be tardy and incur cost w_j.

Although the above description is based upon building up a schedule from the beginning to the end, the dynamic programming procedure defined below works by calculating values of the cost function starting at the end of the schedule and working backwards in time. The function $f(j, t, d)$ will represent the minimum cost of scheduling jobs j through n, under the assumption that jobs 1 through $j-1$ have already been placed in a partial schedule that is characterized by two parameters: t, the completion time of the last batch in the schedule, and d, the earliest due date in the last batch of the schedule. Since the procedure works backwards in time, the schedule for the first $j-1$ jobs will not yet have been determined, but evaluating f at all possible values of d and t will cover all the possible preceding partial schedules without having to explicitly construct them. $f(j, t, d)$ is calculated as the minimum of three quantities, corresponding to the three choices described above for adding a job to a partial schedule. The function is defined formally below.

2.1. Dynamic batching procedure (DB-1)

Let the jobs be ordered so that $d_1 \leqslant d_2 \ldots \leqslant d_n$. Define the following function:

$f(j, t, d)$ = minimum cost of scheduling jobs j through n, starting at time t, given that d is the earliest due date in the last batch of a partial schedule for jobs $1 \ldots j-1$.

$$f(j,t,d) = \min \begin{cases} f(j+1,\ t+p_j, d) & \text{if } t+p_j \leqslant d, \\ f(j+1,\ t+\Delta+p_j, d_j) & \text{if } t+\Delta+p_j \leqslant d_j, \\ f(j+1,\ t, d) + w_j, \end{cases} \tag{1}$$

$$f(n+1, t, d) = 0 \quad \text{for all } t \text{ and } d.$$

The three quantities on the right-hand side of Eq. (1) represent the three possible scheduling choices for job j that were described above.

Procedure DB-1 is used to calculate $f(1, \Delta, \infty)$, which represents the minimum cost schedule for jobs $1 \ldots n$, starting at time Δ, given that there is no job (and hence a due date of ∞) in the first batch. We start at time Δ because there will always be a setup required for the first batch. The following result establishes the correctness and the running time of the procedure.

Theorem 3. *The WTB problem can be solved in* $O(n^2 \min\{d_{\max}, (P + n\Delta)\})$ *operations, where* $d_{\max}$ *is the largest due date and* $P = \sum_{i=1}^{n} p_i$ *is the sum of job processing times.*

Proof. The correctness of procedure DB-1 follows immediately from Lemma 2. Since there is always an optimal schedule with jobs arranged in increasing order of due date, at each stage of the algorithm we need only consider whether or not to include job j, and whether to add it to the last batch of the current schedule or to add it to a new batch.

To establish the running time of the procedure, we must determine an upper bound on the number of times that $f(j, t, d)$ is evaluated. The number of possible values for t is bounded above by the largest job due date ($d_{\max}$), since no on-time job can be scheduled after then. It is also bounded above by $P + n\Delta$, which represents the amount of time required to process every job in its own batch. Since j and d can each have at most n distinct values, and since each evaluation of $f(j, t, d)$ is done in constant time, the total running time of the procedure is $O(n^2 \min\{d_{\max}, (P + \Delta)\})$. □

3. WTB with equal processing times

We now consider several restricted versions of the WTB problem. We have already seen (Lemma 1) that when all jobs have the same due date, the problem remains NP-complete. The result below applies when all jobs have the same processing time, in which case the dynamic programming algorithm presented above runs in strongly polynomial time.

Theorem 4. *For the WTB problem with equal job processing times, the running time of procedure DB-1 is* $O(n^4)$.

Proof. When all jobs have processing time p, the value of t at every stage of procedure DB-1 can be expressed in the form $a\Delta + bp$, where a represents the number of batches and b represents the number of jobs in the schedule so far. Since $a, b \in \{1, \ldots, n\}$, t can have at most n^2 distinct values. Since j and d can have n distinct values each, the function $f(j, t, d)$ must be evaluated at most n^4 times, where each evaluation requires a constant number of operations. □

4. WTB with equal weights

In the preceding section it was shown that when the dynamic programming procedure used to solve the general WTB problem in pseudo-polynomial time is applied to the special case in which all job processing times are equal, the procedure runs in polynomial time. We now consider a different version of the problem: the jobs may have arbitrary processing times, but the weights are all equal. Without loss of generality, we will assume all job weights are unity, so the problem becomes that of finding a schedule to minimize the number of tardy jobs. It is easy to see that when procedure DB-1 is applied to this version of the problem, its running time remains pseudo-polynomial. It is nonetheless possible to solve the equal-weight WTB problem in polynomial time, using a dynamic programming algorithm based upon a different recurrence relation.

Suppose the jobs are ordered by increasing due date, so that $d_1 \leqslant d_2 \cdots \leqslant d_n$. One method for finding a schedule that maximizes the number of on-time jobs is to find the largest $i \in \{1 \ldots n\}$ such that i jobs can be scheduled on-time. In order to schedule i jobs on time, there must be t jobs in the last batch and $i - t$ jobs in earlier batches, for some t in $\{1,\ldots, i\}$. Minimizing the completion time of such a schedule can be accomplished by minimizing the completion times of these two components: the last batch of t jobs, and the preceding schedule for $i - t$ jobs. If it is known that the earliest due date in the last batch is d_k, then by Lemma 2, which states that there is an optimal schedule with EDD ordering, the $i - t$ jobs in the batches preceding the last batch must be chosen from the set $\{1,\ldots, k-1\}$. Furthermore, the t jobs in the last batch must be chosen from among the jobs with index k or greater. We now define the following functions to capture these ideas.

Let $h(i, k, j)$ be the minimum processing time of a batch which contains job k and $i - 1$ jobs chosen from $\{k+1,\ldots, j\}$, for a total of i jobs. If there are fewer than i jobs in the set $\{k,\ldots, j\}$, let $h(i, k, j) = \infty$; otherwise, the minimum processing time of such a batch will be given by $\Delta + p_k + P_{i-1}(k+1, j)$, where the last term represents the sum of the $i - 1$ smallest processing times in $\{p_{k+1},\ldots, p_j\}$.

Let $g(i, j)$ be the minimum completion time of a schedule with i on-time jobs chosen from the set $\{1,\ldots, j\}$.

Let $f(i, j, k, t)$ be the minimum completion time of a schedule with i on-time jobs chosen from the set $\{1,\ldots, j\}$, in which d_k is the earliest due date in the last batch, and there are t jobs in the last batch (including job k).

As explained above, the value of $f(i, j, k, t)$ can be calculated by minimizing the completion times of its two components: the last batch of t jobs, and the preceding schedule for $i - t$ jobs. From the function definitions above, we see that these two quantities are simply $h(t, k, j)$ and $g(i - t, k - 1)$, respectively. Thus $f(i, j, k, t)$ will be equal to the sum of these two values, if the sum is not larger than d_k, the earliest due date in the last batch. If the sum exceeds d_k, then $f(i, j, k, t)$ is set equal to ∞, indicating that there is no feasible schedule that satisfies the specified requirements.

Defining the recurrence relation for $g(i, j)$ is simply a matter of observing that $g(i, j)$ is the minimum of the function $f(i, j, k, t)$ over all possible choices of k and t.

The formal definitions for the three functions at the heart of the dynamic programming procedure are given below.

4.1. Procedure DB-2

Let the jobs be ordered so that $d_1 \leqslant d_2 \ldots \leqslant d_n$.

$$h(i,j,k) = \begin{cases} \Delta + p_k + P_{i-1}(k+1, j) & \text{if } j - k + 1 \geqslant i, \\ \infty & \text{otherwise.} \end{cases}$$

$$g(i,j) = \min_{1 \leqslant k \leqslant j,\ 1 \leqslant t \leqslant i} f(i,j,k,t).$$

$$g(0, j) = 0 \quad \text{for all } j.$$

$$f(i, j, k, t) = \begin{cases} g(i - t,\ k - 1) + h(t, k, j) & \text{if } g(i - t,\ k - 1) + h(t, k, j) \leqslant d_k, \\ \infty & \text{otherwise.} \end{cases}$$

These recursive functions are used to calculate $\max_{1 \leqslant i \leqslant n}\{i : g(i, n) < \infty\}$, which represents the maximum number of on-time jobs (and hence the minimum number of tardy jobs) that can be scheduled.

It is easy to verify (through the preceding explanation) that the functions are defined correctly. The following result establishes an upper bound on the running time of the procedure.

Theorem 5. *Procedure DB-2 calculates the minimum number of tardy jobs in* $O(n^4)$ *operations.*

Proof. Since each argument of the function $f(i, j, k, t)$ can take on at most n values, f must be evaluated at most n^4 times. Each evaluation requires calculating the values of the functions g and h. By maintaining the minimum (over all values of k and t) of $f(i, j, k, t)$ and updating it after each evaluation, $g(i, j)$ will be available without any further computation. There are many possible ways of evaluating the function h; one is as follows: For each pair (k, j), first calculate $h(1, k, j)$, which is simply $\Delta + p_k$. Next sort the processing times in $\{p_{k+1}, \ldots, p_j\}$. By starting at the beginning of the list and repeatedly adding the next processing time to the current value of h, it is possible to evaluate $h(i, k, j)$ for each value of $i \in \{1, \ldots, j - k + 1\}$. Since sorting can be done in $O(n \log n)$ time and must be done for the $O(n^2)$ possible (i, j) pairs, it is possible to calculate *all* values of $h(i, k, j)$ in $O(n^3 \log n)$ time. Thus the total running time for procedure DB-2 is $O(n^4)$. □

5. Agreeable deadlines: An $O(n \log n)$ algorithm

In the preceding sections we presented polynomial algorithms for two restricted versions of the WTB problem; one with equal job weights, and one with equal job processing times. By further restricting these special cases, it is possible to solve the problems using a substantially faster algorithm. In particular, when all job weights are equal, we will call the job due dates *agreeable* if, for any two jobs i and j, if $p_i > p_j$ then $d_i \leqslant d_j$. Similarly, when all job processing times are equal, we will call the job due dates *agreeable* if, for any two jobs i and j, if $w_i < w_j$ then $d_i \leqslant d_j$. Referring to these two special cases as the agreeable due date equal weight (ADEW) problem, and the agreeable due date equal processing time (ADEP) problem, respectively, we have the following result.

Lemma 6. *In any instance of either the ADEW or the ADEP problem, let k be the maximum number of jobs that can be completed on-time. Then the schedule in which the k jobs with the latest due dates are on-time is optimal.*

Proof. Given any schedule S which has k on-time jobs, replace every job that does not have one of the k latest due dates with a job that does have one of the k latest due dates but is not among the on-time jobs in S. In the ADEW problem, jobs are replaced by others with the same weight but with smaller processing times and later due dates. In the ADEP problem, jobs are replaced by others with the same processing times but larger weights and later due dates. In both cases, the resulting schedule has k on-time jobs and the weighted number of tardy jobs is no larger than it was for S. Thus there is always an optimal schedule in which the on-time jobs are those with the latest due dates. □

This result suggests a simple method for finding an optimal solution for the ADEP and ADEW problems–use binary search in the range $1, \ldots, n$ to find the largest k such that the k jobs with the latest due

dates can all be scheduled on-time. For each value of k, determining whether or not the k jobs can be scheduled on-time can be done efficiently, as the following lemma indicates.

Lemma 7. *Given k jobs ordered by increasing due date, the question "can all k jobs be completed on-time?" can be answered with* $O(k)$ *operations.*

Proof. The question can be answered as follows. For convenience, renumber the jobs so that $d_1 \leqslant d_2 \leqslant \cdots \leqslant d_k$. Since the jobs start out in the correct order, this renumbering can be done in linear time. Initialize the schedule with a single batch. Starting with job 1, for each job $i \in \{1, \ldots, k\}$, if jobs $1, \ldots, i$ can be completed on-time by adding i to the last batch, then do so. If not, but jobs $1, \ldots, i$ can be completed on-time by starting a new batch containing only job i, then do so. If neither, then there is no way that jobs $1, \ldots, k$ can be completed on-time. By considering one job at a time in this way, the question can be answered after at most k tests. By maintaining the value t, corresponding to the completion time of the last job in the schedule so far, and the value d, corresponding to the earliest due date in the last batch of the schedule so far, each test can be performed in a constant number of operations. □

With the two preceding lemmas, we can establish the following result.

Theorem 8. *The ADEW problem and the ADEP problem can both be solved in* $O(n \log n)$ *operations.*

Proof. Sorting the n jobs can be done in $O(n \log n)$ time. The binary search for the value of k, the number of on-time jobs, requires checking $O(\log n)$ distinct values, where each check can be done in $O(n)$ time (by Lemma 7). Thus the total running time is $O(n \log n)$. □

Finally, we note that when the WTB problem has jobs with equal weights *and* equal processing times, it is simply a special case of the ADEP and the ADEW problems and can therefore be solved using the preceding algorithm.

6. Conclusion

It is interesting to view the WTB problem as a generalization of the classical scheduling problem of minimizing the tardy task weight for a set of jobs. The latter problem is identical to the version of WTB in which the batch setup time is zero. A natural question to ask in this situation is whether or not a non-zero setup time makes the problem harder. Consider, for example, the problem of finding a schedule of batches to minimize the weighted sum of job completion times. When the batch setup time is zero, the problem reduces to a well-known sequencing problem which is easily solved by arranging the jobs so that the ratios p_i/w_i are in increasing order. When there is a non-zero setup time however, the batching problem is NP-complete (see [1]). In contrast, the results in this paper indicate that a non-zero batch setup time does not increase the complexity of the WTB problem. Like the tardy task weight problem, the general version of WTB is NP-complete but can be solved in pseudo-polynomial time, and the restricted versions with equal job processing times or equal job weights are solvable in polynomial time.

Although the restricted versions of WTB with agreeable due dates can be solved in $O(n \log n)$ time by a very simple algorithm, the dynamic programming procedures presented here, though polynomial, have rather large running times. A question for future research is whether or not the running times of these algorithms can be reduced.

References

[1] S. Albers and P. Brucker, "The complexity of one-machine batching problems", *Discrete Appl. Math.* **47**, 87–107 (1993).
[2] P. Brucker, "Scheduling problems in connection with flexible production systems", in: *Proc. 1991 IEEE Int. Conf. on Robotics and Automation*, pp. 1778–1783, 1991.
[3] J. Bruno and P. Downey, "Complexity of task sequencing with deadlines, set-up times and changeover costs", *SIAM J. Comput.* **7**, 393–404 (1978).
[4] E. Coffman Jr., A. Nozari and M. Yannakakis, "Optimal scheduling of products with two subassemblies on a single machine", *Oper. Res.* **37**, 426–436 (1989).
[5] E. Coffman Jr., M. Yannakakis, M. Magazine and C. Santos, "Batch sizing and sequencing on a single machine", *Ann. Oper. Res.* **26**, 135–147 (1990).
[6] G. Dobson, U.S. Karmarkar and J.L. Rummel, "Batching to minimize flow times on one machine", *Management Sci.* **33**, 784–799 (1987).
[7] R.M. Karp, "Reducibility among combinatorial problems", in: R.E. Miller and J.W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, New York, 1972, pp. 85–103.
[8] E. Lawler and J. Moore, "A functional equation and its application to resource allocation and sequencing problems", *Management Sci.* **16**, 77–84 (1969).
[9] D. Naddef and C. Santos, "One-pass batching algorithms for the one machine problem", *Discrete Appl. Math.* **21**, 133–145 (1988).
[10] D. Shallcross, "A polynomial algorithm for a one machine batching problem", *Oper. Res. Lett.* **11**, 213–218 (1992).